GPT2 · AI Chat with Source Viewer – Project Report

1. Project Overview
GPT2 · AI Chat with Source Viewer is a Retrieval-Augmented Generation (RAG) application available in two modes:

- **Interactive Google Colab notebook** – supports multiple large language models (Gemini via OpenRouter, OpenRouter owl-alpha, local GPT-2) and stores the vector index in Google Drive for persistence.
- **Deployable web application** – a Flask-based web UI with Firebase Authentication, real-time chat memory, multi-file upload, and source chunk inspection. It runs inside a Docker container and deploys easily on Hugging Face Spaces.

Users upload documents (PDF, TXT, DOCX, CSV, images with OCR), and the system retrieves the most relevant chunks using semantic search. Answers are generated from the chosen language model (or a fallback model), complete with a source viewer that shows which document sections were used.

---

2. Key Features

🔐 User Authentication (Web App)
- **Email/Password Login & Sign-up** powered by Firebase Auth.
- Users must authenticate before accessing the chat interface.
- Logout functionality.

💬 Persistent Chat Memory (Web App)
- All messages (user questions + bot answers) are stored in Firebase Realtime Database per user.
- Chat history is loaded instantly on login.
- “New Chat” button clears only the visual display, preserving history in the database.

📂 Multi-File & Large File Upload (Both Modes)
- **Multiple files at once** supported (PDF, TXT, DOCX, CSV, PNG/JPG with OCR).
- Custom upload button with file-name display (web version).
- Backend handles files up to **100 MB** (Flask `MAX_CONTENT_LENGTH`).
- Error handling for oversized files.

🧠 Multi-Model RAG Engine
The system can use one of three language model backends:

Model	Provider / Route	Notes
Gemini	OpenRouter (google/gemini-... models)	Free tier, with automatic fallback across multiple Gemini variants if rate-limited.
OpenRouter owl-alpha	OpenRouter (openrouter/owl-alpha)	Direct OpenRouter model, reliable free tier.

| Local GPT-2 | Hugging Face Transformers (gpt2) | Runs entirely inside the container, no API key needed. |

The Colab notebook offers interactive selection; the web app can be configured to use any of the three via a dropdown or environment variable.

🔍 Source Chunk Viewer
- After each answer, a “View source chunks” button shows the exact text snippets retrieved.
- Up to 3 chunks displayed, truncated to 350 characters.

💾 Persistent Vector Store (Colab)
- The in-memory vector store (embeddings + documents) can be saved to Google Drive as a pickle file.
- On a new Colab session, users can reload the saved store or upload a fresh document.

🎨 Modern Web UI
- Dark theme, animated typing indicator, blurred input bar.
- Sidebar with **New Chat** and **Logout**.
- Fully responsive design.

🐳 Docker & Hugging Face Spaces Deployment
- Lightweight Docker image (Python 3.9-slim) with Tesseract OCR.
- Port 7860 exposed (HF Spaces default).
- Environment variable `OPENROUTER_KEY` used for OpenRouter API (safe for public repositories).

---

3. Technical Architecture

Backend (Flask / Colab)
Component	Technology / Model
Web Framework	Flask
CORS	flask-cors
Embedding	`sentence-transformers/all-MiniLM-L6-v2`
Text Extraction	PyPDF2, python-docx, csv, Pillow, pytesseract
Vector Storage	Custom in-memory store (cosine similarity)
Persistent Storage	Pickle file (Google Drive in Colab)
Generation Models	OpenRouter (Gemini, owl-alpha), local GPT-2
File Handling	werkzeug secure_filename, multipart form data
Maximum Upload Size	100 MB

API Endpoints (Web App):
- `GET /` → Serves `index.html`
- `POST /upload` → Accepts multiple files, indexes them, returns chunk counts.
- `POST /chat` → Takes query + model choice, retrieves chunks, calls selected model, returns answer + source chunks.

Colab Flow:
- Install dependencies, load embedding model, create vector store.
- Upload document → ingest & index.
- Choose LLM (Gemini/OpenRouter/Local) → looped Q&A with formatted source display.
- Optionally save/load store from Google Drive.

Frontend (index.html for Web App)
- Single HTML file with embedded CSS/JS.
- Firebase Auth (email/password), Realtime Database for chat history.
- Multi-file upload via `fetch` + `FormData`.
- Dynamic message bubbles, typing animation, source toggle.
- Model selection dropdown (GPT-2 / Gemini / OpenRouter).

---

4. Development Progress & Key Milestones

Phase 1 – Core RAG Pipeline
- Implemented document text extraction for PDF, DOCX, TXT, CSV, PNG/JPG (OCR).
- Text chunking with configurable size (500 chars) and overlap (50 chars).
- Integrated `all-MiniLM-L6-v2` for embedding; cosine similarity retrieval.
- Added local GPT-2 answer generation with basic prompting.

Phase 2 – Chat Interface & Source Viewer
- Designed the dark-themed UI with user/bot bubbles.
- Added typing indicator during backend processing.
- Built the “View source chunks” toggle to inspect retrieved context.

Phase 3 – Authentication & Persistent Memory (Web)
- Added Firebase Authentication (email/password).
- Implemented chat history storage in Firebase Realtime Database.
- Automatic history loading and “New Chat” reset.

Phase 4 – Multi-Model Support (OpenRouter & Gemini)
- Integrated OpenRouter API with multiple Gemini fallback models.
- Added `openrouter/owl-alpha` as a second cloud option.
- Created a unified `answer_fn` architecture so any model can be plugged in.
- Added environment variable support for `OPENROUTER_KEY`.

Phase 5 – Multi-File Upload & Large File Handling
- Frontend: multiple file selection, file name display.
- Backend: accepted list of files, indexed each, returned summary.
- Increased Flask upload limit to 100 MB with custom 413 error handler.

Phase 6 – Persistent Vector Store (Colab)
- Implemented `pickle` serialization of the vector store.
- Added Google Drive integration to save/load the store between Colab sessions.

- Provided user choice to reuse old store or upload new documents.

Phase 7 – Repetition & Output Quality Fixes
- GPT-2 was looping (e.g., “Fake News Detection” repeating).
- Fixes: increased tokenizer max length to 1536, enabled sampling (temp=0.7), added repetition_penalty=1.2.
- Improved answer extraction by splitting on “Answer:” and taking the last segment.

Phase 8 – Dockerisation & Hugging Face Spaces Deployment
- Created `Dockerfile` with all dependencies (including Tesseract).
- Wrote `requirements.txt`.
- Tested locally and deployed on Hugging Face Spaces using the Docker SDK.
- Application now publicly accessible via `*.hf.space`.

---

5. Challenges & Solutions

Challenge	Solution
GPT-2 repetitive output	Increased context window, used sampling with repetition penalty, improved prompt parsing.
Large file uploads timing out	Set 100 MB cap, chunked embeddings, streaming uploads.
Multiple cloud models & rate limits	Built fallback list inside Gemini function; retry after 5s on 429 errors.
Persisting vector store across sessions	Used pickle + Google Drive (Colab); saved store can be reloaded without re-indexing.
Firebase security for user data	Recommended Realtime Database rules: read/write only for authenticated users.
Deployment on HF Spaces with env key	Stored `OPENROUTER_KEY` as a Space secret; backend reads from `os.environ`.

---

6. Conclusion
The **GPT2 · AI Chat with Source Viewer** is a versatile, production-ready RAG system that can be used as a standalone Colab notebook or as a containerised web application. It supports multiple document formats, allows users to choose between free cloud LLMs (Gemini, OpenRouter) or a fully local GPT-2, and provides full source transparency. With Firebase authentication and real-time chat history, it offers a personalised experience. The Dockerised version deploys instantly on Hugging Face Spaces, making it accessible to anyone with a browser.

Future Improvements
- Replace in-memory vector store with a persistent database (e.g., ChromaDB, FAISS).
- Add streaming token output for a more interactive feel.
- Support for more file formats (doc, xlsx, epub).

- User dashboard to manage uploaded documents and chat sessions.
- Improved answer evaluation metrics and feedback collection.